Data Structures

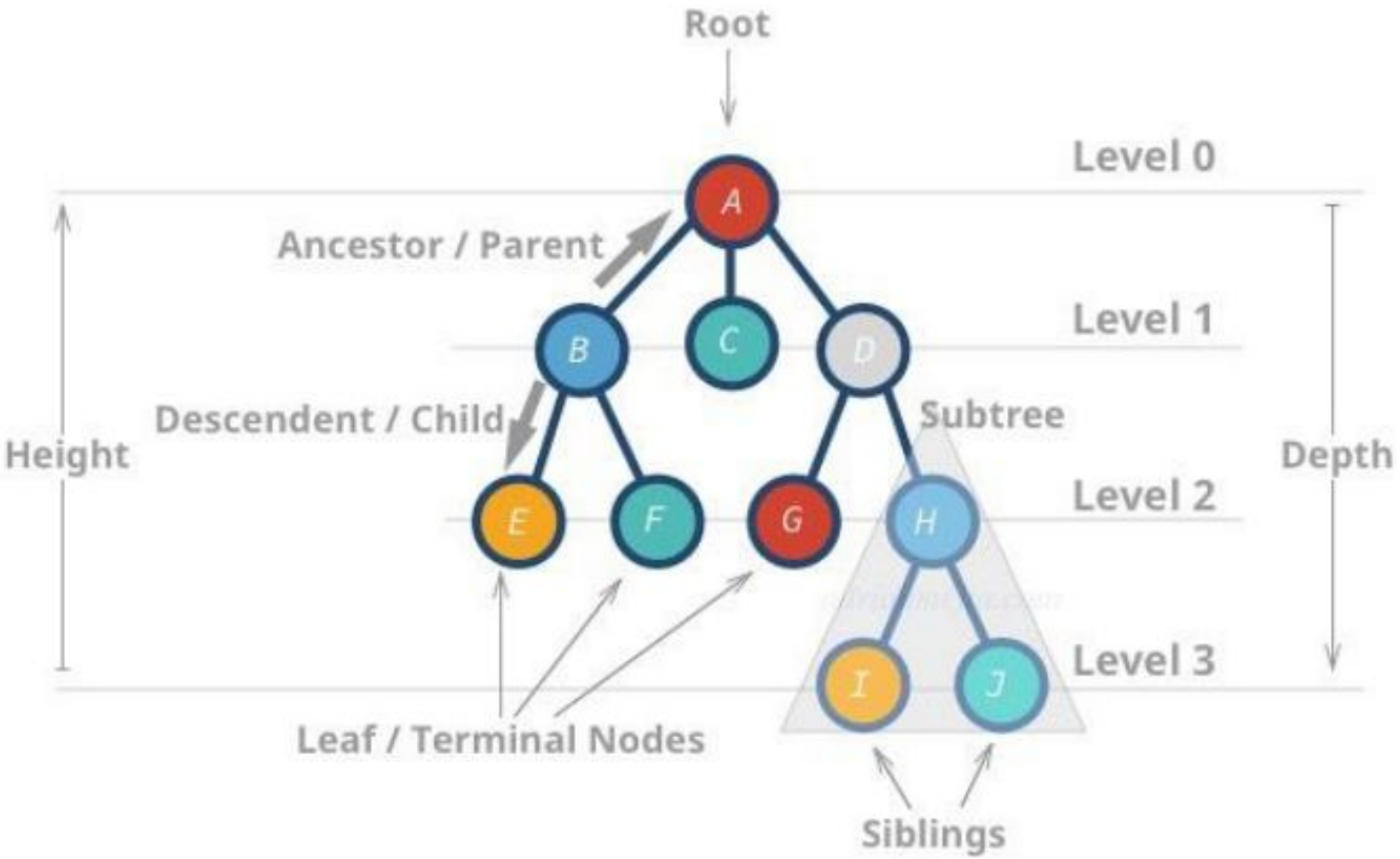
Trees

Tree Data Structure

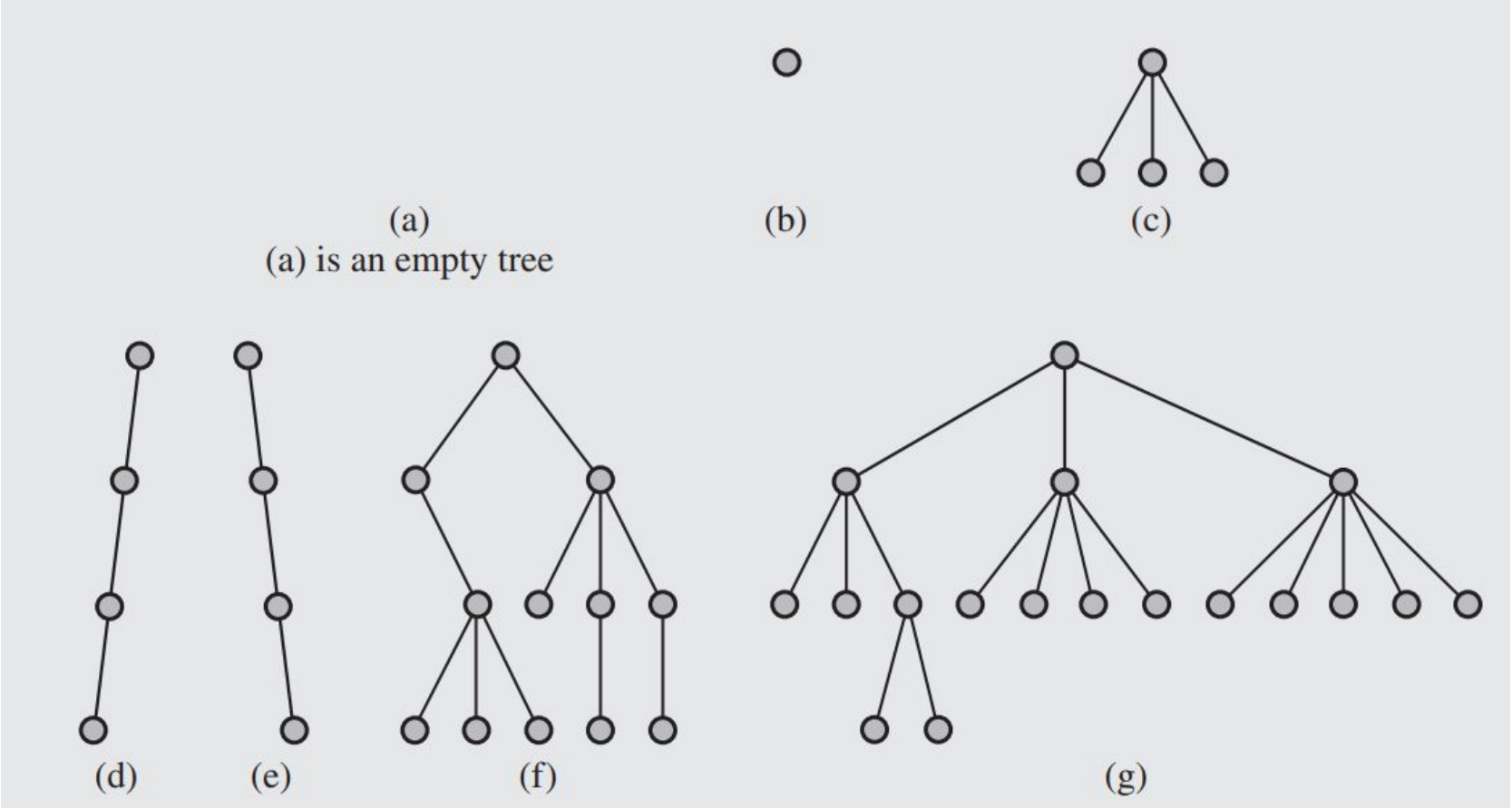
- A tree is a **non linear** data structure that simulates a hierarchical tree structure with a **root** value and **sub trees** of children with parent node, represented as set of linked nodes.

- **Root –** Root is a special node in a tree. The entire tree is referenced through it. It does not have a parent.
- **Parent Node –** Parent node is an immediate predecessor of a node.
- **Child Node –** All immediate successors of a node are its children.
- **Siblings –** Nodes with the same parent are called Siblings.
- **Leaf –** Last node in the tree. There is no node after this node.
- **Edge –** Edge is a connection between one node to another. It is a line between two nodes or a node and a leaf.
- **Path –** Path is a number of successive edges from source node to destination node.

Terminologies



Examples of trees



Tree Properties

- Tree can be termed as a **RECURSIVE** data structure.
- In a valid tree for **N Nodes** we have **N-1 Edges/Links**
- **Depth of Node –** Depth of a node represents the number of edges from the tree's root node to the node.
- **Height of Node –** Height of a node is the number of edges on the longest path between that node & a leaf.
- **Height of Tree –** Height of tree represents the height of its root node.

Binary Tree

- A binary tree is a tree data structure in which each node has **at most two children**, which are referred to as the left child(LC) and the right child(RC).
- **Binary Tree Terms & Properties**
  - A binary tree is called STRICT/PROPER/Full binary tree when each node has 2 or 0 children.
  - A binary tree is called COMPLETE binary tree if all levels except the last are completely filled and all nodes are as left as possible.
  - Max number of nodes at a given level 'x' = 2^x
  - For a binary tree, maximum number of nodes with height 'h' = 2^(h+1) – 1
  - A binary tree is called BALANCE binary tree, if the difference between the height of left subtree and right subtree for every node is not more than k(usually k=1)

Traversal Techniques

- Tree traversal also known as tree search and walking the tree refers to the process of visiting (checking and/or updating each node in a tree data structure, exactly once. Such traversals are classified by the order in which the nodes are visited.
- **Depth-first search/traversal (DFS)** These searches are referred to as depth-first search (DFS), since the search tree is deepened as much as possible on each child before going to the next sibling.
  - Pre-order (NLR)
  - In-order (LNR)
  - Post-order (LRN)
- **Breadth-first search/traversal (BFS)** Trees can also be traversed in level-order, where we visit every node on a level before going to a lower level. This search is referred to as breadth-first search (BFS), as the search tree is broadened as much as possible on each depth before going to the next depth.

Pre-order (NLR) (node - left-right)

- Access the data part of the current node.
- Traverse the left subtree by recursively calling the pre-order function.
- Traverse the right subtree by recursively calling the pre-order function.

Pre-order (Pseudocode)

```
void printPreorder(TreeNode* r)
{
    IF r==NULL THEN   return
    PRINT r value
    printPreorder(r left)
    printPreorder(r right)
}
```

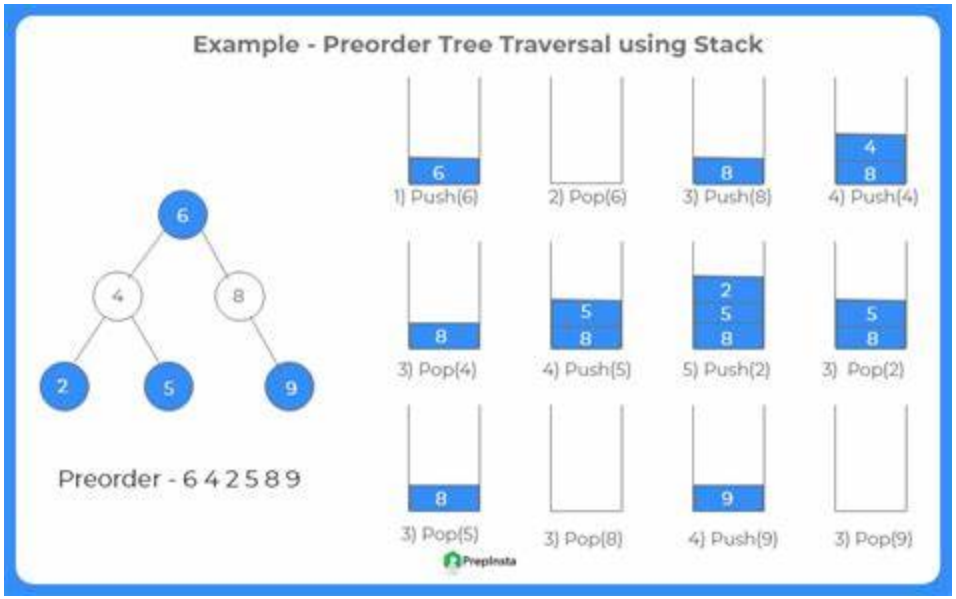
```cpp
void preorderIterative(TreeNode* root) {
    if (root == nullptr) return;

    stack<TreeNode*> stack;
    stack.push(root);

    while (!stack.empty()) {
        TreeNode* current = stack.top();
        stack.pop();

        cout << current->value << " "; // Visit root

        // Push right child first so that left child is processed next
        if (current->right) {
            stack.push(current->right);
        }
        if (current->left) {
            stack.push(current->left);
        }
    }
```



In-order (LNR) (left - node - right)

- Traverse the left subtree by recursively calling the in-order function.
- Access the data part of the current node.
- Traverse the right subtree by recursively calling the in-order function.
  - In BST in-order traversal retrieves the keys in ascending sorted order.

In-Order (Pseudocode)

```
void printinorder(TreeNode*r)
{
    IF r==NULL THEN   return
    printlnorder(r left)
    PRINT r value
    printlnorder(r right)
}
```

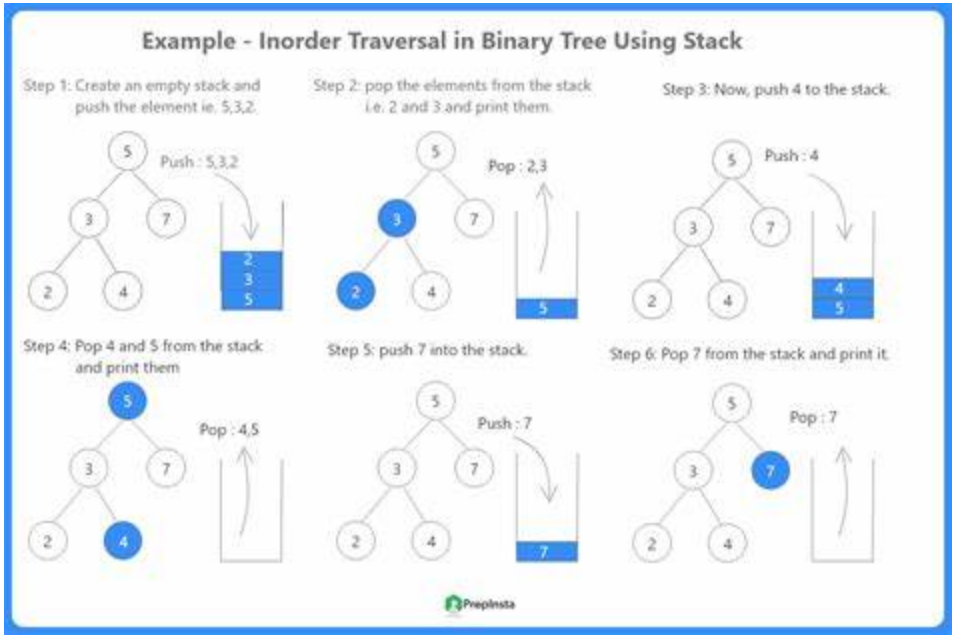
Inorder Traversal (Loop)

```cpp
void inorderIterative(TreeNode* root) {
    stack<TreeNode*> stack;
    TreeNode* current = root;

    while (current != nullptr || !stack.empty()) {
        // Reach the leftmost node of the current node
        while (current != nullptr) {
            stack.push(current);
            current = current->left;
        }

        // Current must be nullptr at this point
        current = stack.top();
        stack.pop();
        std::cout << current->value << " "; // Visit root

        // We have visited the node and its left subtree.
        // Now, it's the right subtree's turn.
        current = current->right;
    }
}
```



Post-order (LRN) (left-right - node)

- Traverse the left subtree by recursively calling the post-order function.
- Traverse the right subtree by recursively calling the post-order function.
- Access the data part of the current node.

Post-Order (Pseudocode)

```
void printPostorder(TreeNode* r)
{
    IF r==NULL THEN  return
    printPostorder(r left)
    printPostorder(r right)
    PRINT r value
}
```

```cpp
void postorderIterative(TreeNode* root) {
    if (root == nullptr) return;

    stack<TreeNode*> stack1;
    stack<TreeNode*> stack2;

    stack1.push(root);

    // First stack processes nodes in root-right-left order
    while (!stack1.empty()) {
        TreeNode* current = stack1.top();
        stack1.pop();
        stack2.push(current);  // Push to second stack

        // Push left first so right is processed first in the second stack
        if (current->left) {
            stack1.push(current->left);
        }
        if (current->right) {
            stack1.push(current->right);
        }
    }

    // Now process the second stack which is in left-right-root
    while (!stack2.empty()) {
        std::cout << stack2.top()->value << " ";
        stack2.pop();
    }
}
```

